Introduction

Introduction I

This `template_code_project` serves as the foundational exemplar for the Research Project Template ecosystem, demonstrating a fully-tested numerical optimization implementation bracketed by rigorous infrastructure, hermetic testing, and extensive documentation architectures. The prose, the labelled figures, and the labelled equations have all been generated through an auditable custody chain starting from algorithm implementation through strict CI/CD validation to multi-format `.pdf` compilation.

Template Architecture Context I

Scientific engineering requires mathematical accuracy combined with software reliability. This project unifies theoretical optimization with the repository's three foundational pillars:

1. **`infrastructure/` Layer (Root Directory)**: A modular stack of importable Python packages providing the computational scaffolding. The current package count is measured in the template repository's generated canonical facts rather than repeated here because it changes as infrastructure modules are added or retired.
2. **`tests/` Framework (`projects/templates/template_code_project/tests/`)**: An uncompromising validation layer maintaining a zero-mock testing policy. This is enforced automatically via the CI workflow mapping to `pyproject.toml` directives.

Template Architecture Context II

3. **`docs/` Knowledge Base (`projects/templates/template_code_project/docs/`)**: A structured repository of architectural guidelines, operational patterns, and the Rigorous Agentic Scientific Protocol (RASP) that governs the AI-assisted agents writing these very texts.

This implementation of gradient descent algorithms for solving optimization problems is used as the vehicle to demonstrate these pillars. The theoretical problem stated in [@eq:optimization_problem] is mapped programmatically inside the optimizer module:

$$\min_{x \in \mathbb{R}^n} f(x) \tag{1}$$

where $f : \mathbb{R}^n \to \mathbb{R}$ is a continuously differentiable objective function.

Infrastructure Integration I

Rather than existing as isolated scripts, this project extensively leverages the `infrastructure` layer:

- **Scientific Utilities**: Utilizing `infrastructure.scientific.stability` and `infrastructure.scientific.benchmarking` to check numerical boundaries and exercise runtime performance diagnostics without pinning host-dependent timings.
- **Hermetic Validation**: Deploying `infrastructure.validation` components (`markdown_validator`, `output_validator`) to ensure generated artifacts are structurally valid and traceable.
- **Reporting & Rendering**: Employing `infrastructure.rendering.pdf_renderer` and `infrastructure.reporting.executive_reporter` to automatically transform code outputs into this finalized manuscript.

Algorithm Overview I

The reference gradient descent algorithm iteratively updates the solution using the rule shown in [@eq:gradient_descent_update]:

$$x_{k+1} = x_k - \alpha \nabla f(x_k) \tag{2}$$

where $\alpha > 0$ is the step size (learning rate) and $\nabla f(x_k)$ is the gradient of the objective function at iteration k.

Exemplar Implementation Goals I

As the representative project for the repository, this implementation explicitly demonstrates:

1. **Infrastructure-Coupled Code**: Scientific implementations that delegate logging, file ops, and reporting to the `infrastructure` core.
2. **Zero-Mock Verification**: A strict comprehensive validation suite proving numerical accuracy without artificial test boundaries.
3. **Automated Research Pipelines**: High-precision analyses that generate publication-quality, accessible visualizations automatically.
4. **Agentic Documentation standards**: Native adherence to the RASP methodology and `AGENTS.md` guidelines, ensuring the logic remains verifiable by both human and artificial intelligence.

Reader's guide to the manuscript I

- **[@sec:methodology]** ties pseudocode to `gradient_descent()` and explains how stability checks and benchmarks call into `infrastructure.scientific`.
- **[@sec:results]** is figure-centric: every panel references a generator in `src/figures/` (orchestrated via `scripts/optimization_analysis.py`) and uses `{{CONFIG_*}}` / `{{RESULT_*}}` placeholders for numeric values.
- **[@sec:experimental_setup]** lists the exact YAML fields (`experiment:` block) that controlled the run whose artifacts you are viewing.
- **[@sec:reproducibility]** records the configuration hash and artifact inventory produced alongside the PDF.
- **[@sec:scope]** states scope and related literature so the exemplar is not mistaken for a general-purpose optimizer benchmark suite.

Why a quadratic model I

Restricting f to a quadratic with known (A, b) keeps the optimum, gradient, and spectral data explicit. For $A = I$ and $b = \mathbf{1}$, the optimal point is $x^* = \mathbf{1}$ and gradient descent with fixed α reduces to a linear iteration in the error (see [@eq:scalar_linear_update] in the results section). That simplicity isolates step-size effects and makes divergent choices ($\alpha \geq 2$ in 1D) visible in both plots and CSV rows without requiring trust-region or line-search fixes—those extensions are left to future work ([@sec:scope]).